SecureText: A Secure Message Communication System Using RSA Public-Key Cryptography, Digital Signatures, and Message Integrity Verification

1. Cover Page

Project Report

SecureText: A Secure Message Communication System Using RSA Public-Key Cryptography, Digital Signatures, and Message Integrity Verification

Submitted in partial fulfilment of the requirements for the course **Data Security and Privacy / Cryptography and Network Security (DSP)**

Bachelor of Technology in **[YOUR DEPARTMENT]**

Student name	[YOUR NAME]
USN / Register number	[YOUR USN]
College	[YOUR COLLEGE NAME]
University	[YOUR UNIVERSITY]
Department	[YOUR DEPARTMENT]
Guide	[GUIDE NAME AND DESIGNATION]
Academic year	[YYYY-YY]

2. Certificate

This is to certify that the project work entitled **“SecureText: A Secure Message Communication System Using RSA Public-Key Cryptography, Digital Signatures, and Message Integrity Verification”** is a bonafide work carried out by **[YOUR NAME]** (USN: **[YOUR USN]**) in partial fulfilment of the requirements for the course Data Security and Privacy / Cryptography and Network Security of **[YOUR COLLEGE NAME]** during the academic year **[YYYY-YY]**.

Signature of Guide [GUIDE NAME]	Signature of Head of Department [HOD NAME]

Date: [DATE] Place: [PLACE]

3. Declaration

I, **[YOUR NAME]**, USN **[YOUR USN]**, hereby declare that this project report titled “SecureText” is my own work, carried out under the guidance of **[GUIDE NAME]**. External libraries, standards and references used are acknowledged in the References section. This work has not been submitted to any other institution for any degree or course.

Signature: _____ Date: [DATE] Place: [PLACE]

4. Acknowledgement

I thank my guide **[GUIDE NAME]** for guidance and encouragement, the Head of the Department **[HOD NAME]** and the faculty of **[YOUR DEPARTMENT]** for the facilities provided, and my family and friends for their support. I also acknowledge the open-source Python and `cryptography` communities whose documentation made this work possible.

5. Abstract

Public-key cryptography solves the key-distribution problem of symmetric ciphers and enables digital signatures. This project presents **SecureText**, an offline desktop application written in Python 3 with a Tkinter/ttk interface that applies RSA to secure text communication. The application generates RSA key pairs of at least 2048 bits, stores private keys as password-protected PKCS#8 PEM files, encrypts short messages with RSA-OAEP (SHA-256), signs and verifies messages with RSA-PSS (SHA-256), and encrypts messages of any length with a hybrid scheme in which a fresh AES-256-GCM key protects the data and RSA-OAEP wraps the key. It deliberately demonstrates failure cases: decryption with a wrong key, modified ciphertext, modified signed messages, verification with the wrong public key, and altered hybrid envelopes. An educational module explains RSA mathematics with a clearly labelled toy example and explains why textbook RSA is unsafe. All real cryptographic operations use the audited `cryptography` library; no custom algorithm or padding was invented. A suite of 46 automated unit tests was written; in the recorded run on Python 3.13 / Linux all 46 passed. The graphical interface could not be launched in the development sandbox, which lacked Tkinter and a display; its handlers were checked against a simulated Tkinter module and the interface must be exercised manually on the target machine. The project demonstrates confidentiality, integrity and authenticity, and documents its limitations, mainly the absence of certificates and key lifecycle management.

Keywords: RSA, OAEP, PSS, digital signature, AES-GCM, hybrid encryption, key management, Python.

6. Table of Contents

1. Cover Page
2. Certificate
3. Declaration
4. Acknowledgement
5. Abstract
6. Table of Contents
7. Introduction
8. Problem Statement
9. Motivation
10. Aim and Objectives
11. Scope
12. Existing System
13. Limitations of Existing Approaches
14. Proposed System
15. Literature Review
16. Hardware and Software Requirements
17. Feasibility Analysis
18. System Architecture
19. Use-Case Diagram
20. Data-Flow Diagram
21. Application Flowchart
22. RSA Mathematical Foundation
23. RSA Key-Generation Process
24. RSA Encryption and Decryption Algorithms
25. Digital-Signature Algorithm
26. Hybrid-Encryption Workflow
27. Detailed Module Descriptions
28. Implementation Details
29. Important Code Explanations
30. Testing Methodology
31. Test-Case Table
32. Actual Results
33. Screenshots
34. Results and Discussion
35. Security Analysis
36. Advantages
37. Limitations
38. Real-World Applications
39. Future Enhancements
40. Conclusion
41. References
42. Appendix: Installation and Execution

7. Introduction

Messages sent over the Internet or a campus network pass through many devices that a third party could observe or alter. Three properties are normally needed: **confidentiality** (only the intended reader can read it), **integrity** (changes are detected) and **authenticity** (the sender is who they claim to be). Symmetric ciphers such as AES provide fast confidentiality but require both parties to hold the same secret key, which is difficult to establish securely. Public-key cryptography, proposed by Diffie and Hellman [1] and realised practically by Rivest, Shamir and Adleman [2], gives each user a public key that can be freely distributed and a private key that is never shared.

SecureText is a desktop tool that makes these ideas usable and observable. A user generates a key pair, shares the public key, and uses it to encrypt messages that only the private-key holder can read, or signs messages so that anyone can verify them.

8. Problem Statement

To design and implement a secure message communication system that (a) encrypts messages so only the intended recipient can read them, (b) detects any modification of a message, (c) lets a receiver verify who signed a message, (d) handles messages longer than RSA's direct limit, (e) stores private keys safely, and (f) teaches the underlying RSA mathematics, using standard, correctly applied cryptographic primitives rather than home-made algorithms.

9. Motivation

- Many students implement "textbook RSA" with small numbers, which hides the real engineering issues: padding, key sizes, length limits, key storage and error handling.
- Incorrect use of cryptography is a common cause of real-world vulnerabilities. A project that uses established APIs correctly and tests failure cases is a better learning exercise.
- The Module 2 syllabus (public-key principles, RSA, computational aspects, security of RSA, key management) maps directly onto a working application.

10. Aim and Objectives

Aim: Build a working, tested, offline RSA-based secure messaging application with clear documentation.

Objectives

1. Generate RSA key pairs (at least 2048 bits) and export/import public and private keys, the latter password-protected.
2. Encrypt and decrypt short messages with RSA-OAEP using SHA-256, reporting oversize messages clearly.
3. Create and verify digital signatures with RSA-PSS using SHA-256.
4. Encrypt long messages with AES-256-GCM and RSA-OAEP key wrapping in a documented envelope.
5. Demonstrate wrong-key and tampering behaviour.
6. Provide an educational explanation of RSA mathematics and of the risks of textbook RSA.
7. Verify behaviour with automated tests and report results honestly.

11. Scope

In scope: local desktop application; text messages (UTF-8); RSA-2048/3072/4096; file-based key and ciphertext exchange (copy/paste or files); educational module.

Out of scope: network transport, user accounts, certificate authorities, key revocation, file/binary encryption, mobile or web versions, formal security audit.

12. Existing System

- **Plain text communication** (basic chat, SMS, e-mail without protection): readable and modifiable in transit.
- **Symmetric-only tools:** require pre-shared passwords or keys.
- **Mature tools** such as PGP/GPG, S/MIME and TLS: strong, but their complexity hides the mechanics from a learner.
- **Textbook-RSA classroom programs:** use tiny primes and no padding.

13. Limitations of Existing Approaches

Approach	Limitation
Plain text	No confidentiality, integrity or authenticity
Symmetric-only	Key distribution problem; no non-repudiation
Textbook RSA	Deterministic, malleable, tiny keys, no length handling [3]
Professional tools	Not designed for step-by-step demonstration of failures

14. Proposed System

SecureText combines standard primitives behind a graphical interface:

- RSA-OAEP (SHA-256) for direct encryption of short messages.
- RSA-PSS (SHA-256) for signatures.
- AES-256-GCM with RSA-OAEP key wrapping for long messages.
- PKCS#8 PEM with password encryption for private keys; SubjectPublicKeyInfo PEM for public keys.
- Built-in demonstrations of correct failure behaviour and an RSA mathematics tutorial.

15. Literature Review

- **Diffie and Hellman (1976)** introduced public-key cryptography and the idea of digital signatures [1].
- **Rivest, Shamir and Adleman (1978)** presented the RSA cryptosystem based on the difficulty of factoring a product of two large primes [2].
- **Bellare and Rogaway (1994)** introduced Optimal Asymmetric Encryption Padding, adopted in PKCS #1 [4].
- **RFC 8017 (PKCS #1 v2.2)** specifies RSAES-OAEP, RSASSA-PSS and the mathematical primitives [3].
- **NIST SP 800-38D** specifies Galois/Counter Mode, an authenticated encryption mode, and the requirements on nonce uniqueness [5].
- **NIST SP 800-57 Part 1** gives general guidance on key management over the key lifecycle [6].
- **FIPS 180-4** specifies the SHA-2 family, including SHA-256 [7].
- Textbooks such as Stallings [8], Katz and Lindell [9], and Menezes, van Oorschot and Vanstone [10] provide the theoretical background.

16. Hardware and Software Requirements

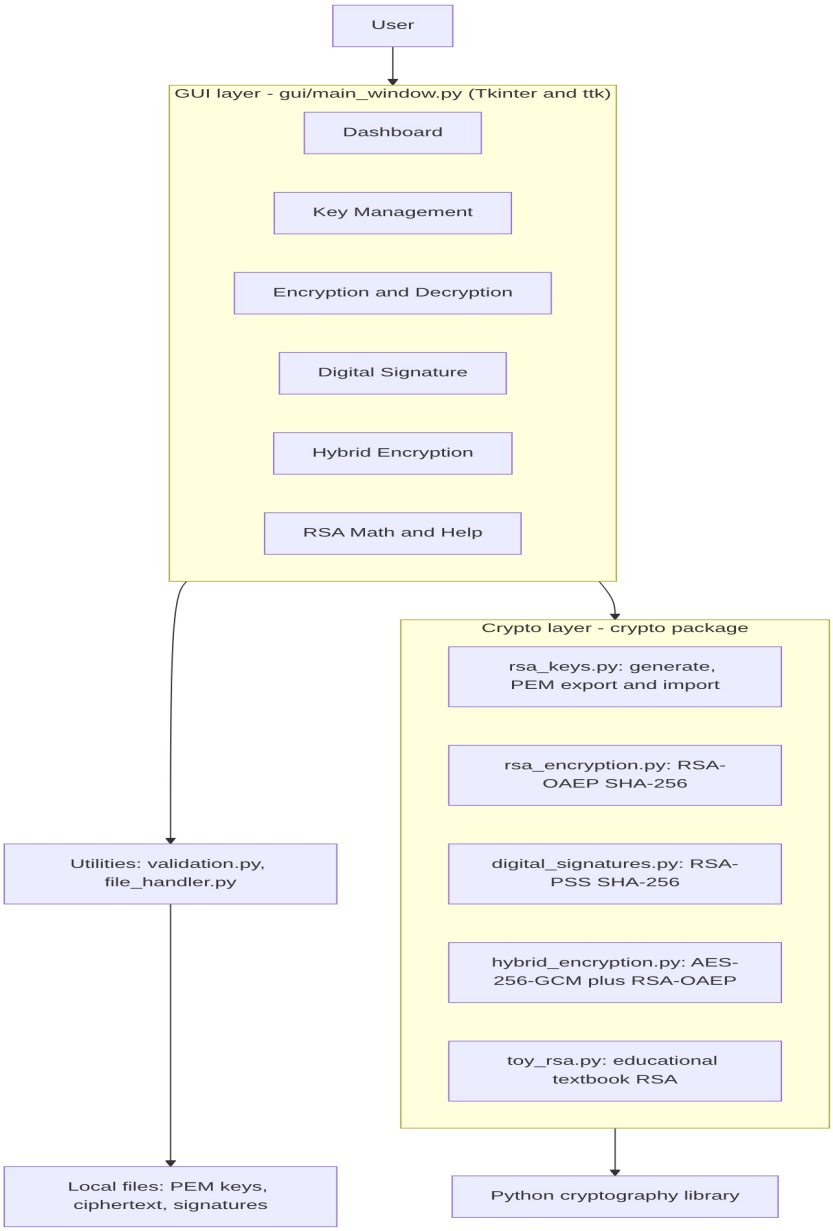
Item	Requirement
Processor	Any modern 64-bit CPU
RAM	2 GB or more
Disk	Under 50 MB
OS	Windows 10/11 (target); Linux and macOS compatible
Python	3.9 or newer (tested on 3.13)
Libraries	`cryptography` (>= 42); Tkinter (bundled with the python.org Windows installer)
Test framework	`unittest` (standard library)
Network	Not required after installation

17. Feasibility Analysis

- **Technical:** Python, Tkinter and `cryptography` are mature and cross-platform.
- **Economic:** all components are free and open source; no cloud or paid API.
- **Operational:** simple tabbed interface; runs offline on a student laptop.
- **Schedule:** the scope (single-machine, five crypto modules, one GUI module) is achievable in a short project period.
- **Legal/ethical:** uses standard algorithms for educational purposes. (Cryptography regulations vary by country; check your institution’s guidance.)

18. System Architecture

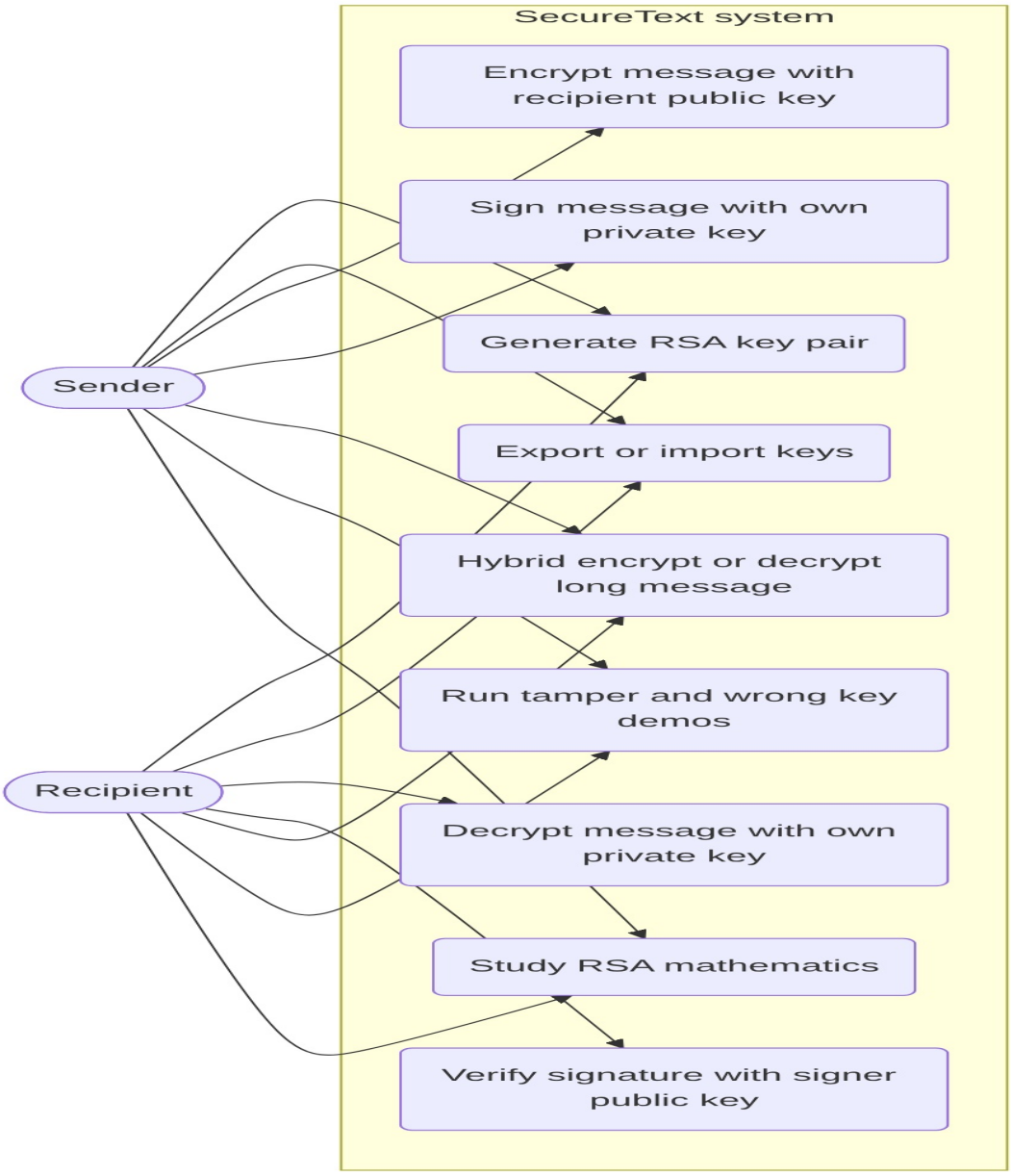
The system has three layers. The GUI contains no cryptographic logic; the crypto package depends only on the `cryptography` library and on validation helpers; utilities handle input checks and files.



System architecture

Mermaid source for all diagrams is in `docs/diagrams/*.mmd` and `docs/diagrams.md`.

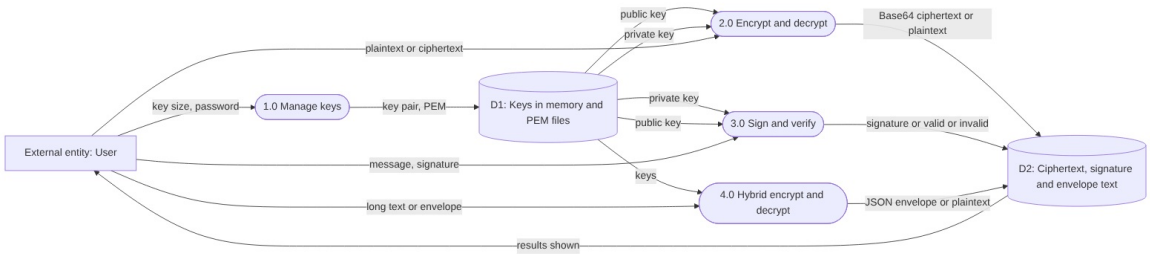
19. Use-Case Diagram



Use cases

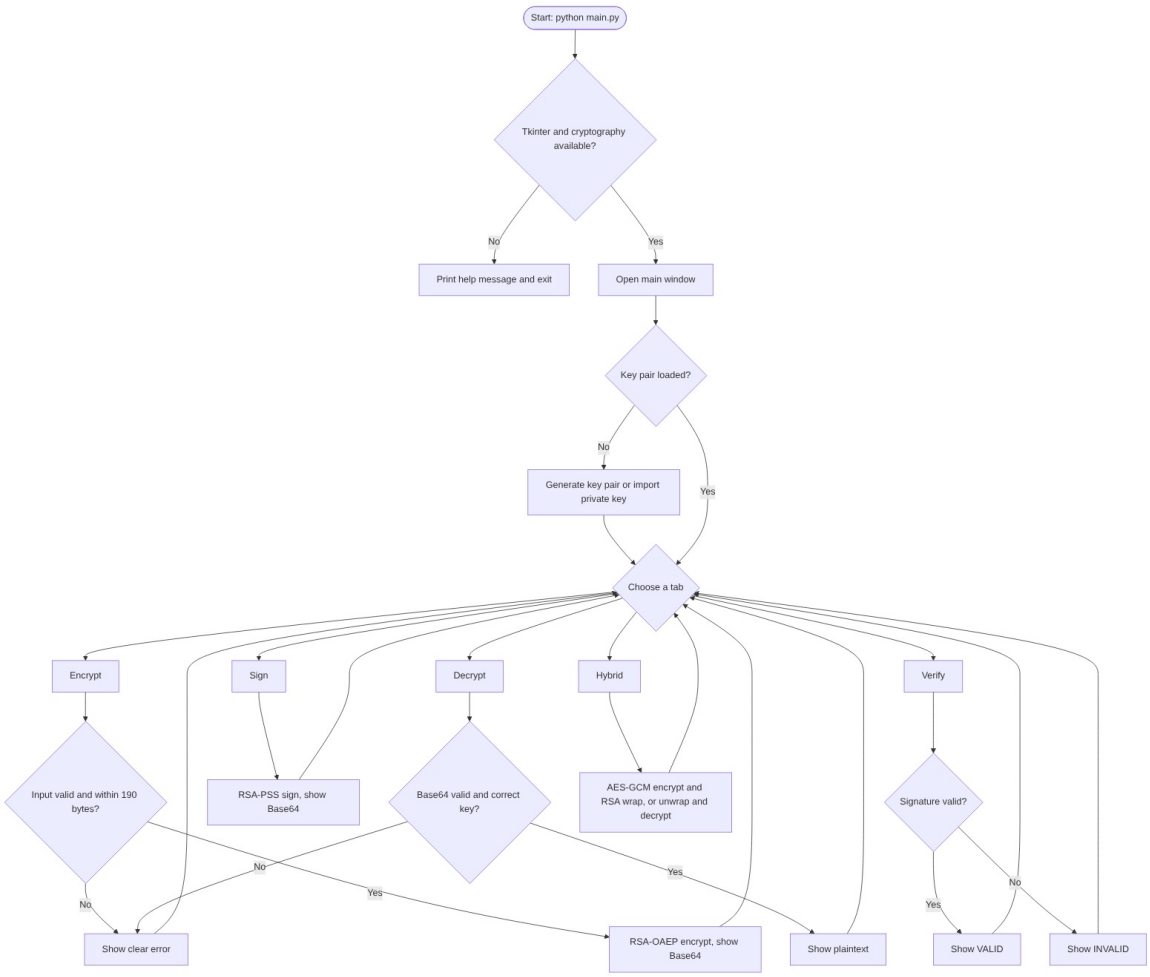
Actors: **Sender** and **Recipient**. Both can generate and manage keys; the sender encrypts and signs, the recipient decrypts and verifies; both can use the hybrid mode, demos and the maths tutorial.

20. Data-Flow Diagram



Level-1 data-flow diagram

21. Application Flowchart



Application flowchart

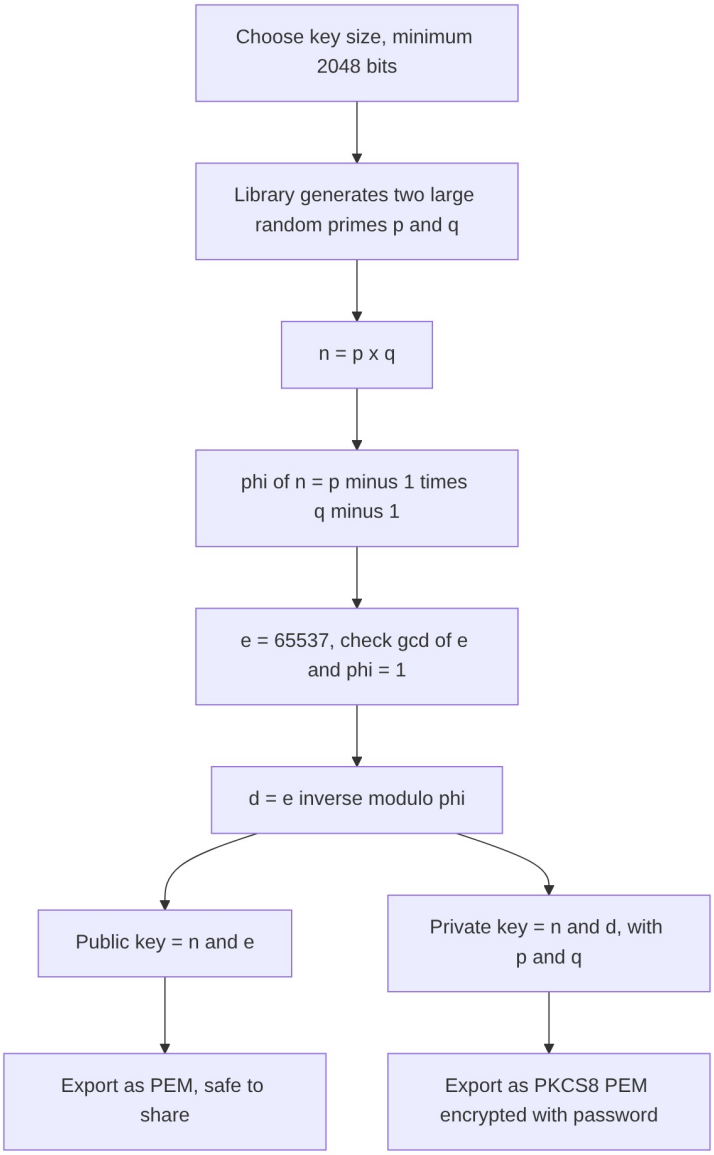
22. RSA Mathematical Foundation

- **Primes p and q:** two large distinct random primes.
- **Modulus:** n = p × q (public).
- **Euler's totient:** φ(n) = (p − 1)(q − 1) (secret). For prime p, φ(p) = p − 1, and φ is multiplicative.
- **Public exponent e:** 1 < e < φ(n) with gcd(e, φ(n)) = 1. SecureText uses 65537.
- **Private exponent d:** d·e ≡ 1 (mod φ(n)), computed with the extended Euclidean algorithm.
- **Correctness:** c = m^e mod n; c^d = m^(ed) = m^(1 + kφ(n)) ≡ m (mod n) by Euler's theorem (and by the Chinese Remainder Theorem when gcd(m, n) ≠ 1).
- **Modular exponentiation:** square-and-multiply reduces mod n at every step, needing about $\log_2(e)$ squarings.
- **Security:** deriving d from (n, e) is as hard as factoring n, as far as is known. This is an assumption, not a proof, and a large quantum computer running Shor's algorithm would break it.

Worked toy example (insecure). p = 61, q = 53 gives n = 3233, φ(n) = 3120. With e = 17, d = 2753 because 17 × 2753 = 46801 = 15 × 3120 + 1. Encrypting m = 65: c = 65^{17} mod 3233 = 2790; decrypting: 2790^{2753} mod 3233 = 65. This example is reproduced by `crypto/toy_rsa.py` and checked by test T44. It is **not** the real implementation: real keys are 2048+ bits and padded.

Why textbook RSA is unsafe: it is deterministic (equal plaintexts give equal ciphertexts), malleable ($E(m_1)\cdot E(m_2) = E(m_1 m_2)$) and leaks small messages when $m^e < n$. Padding schemes OAEP and PSS [3] fix these.

23. RSA Key-Generation Process



Key generation

In code, `rsa.generate_private_key(public_exponent=65537, key_size=...)` from the `cryptography` library generates the primes and exponents; SecureText enforces a minimum of 2048 bits in `utils/validation.py`.

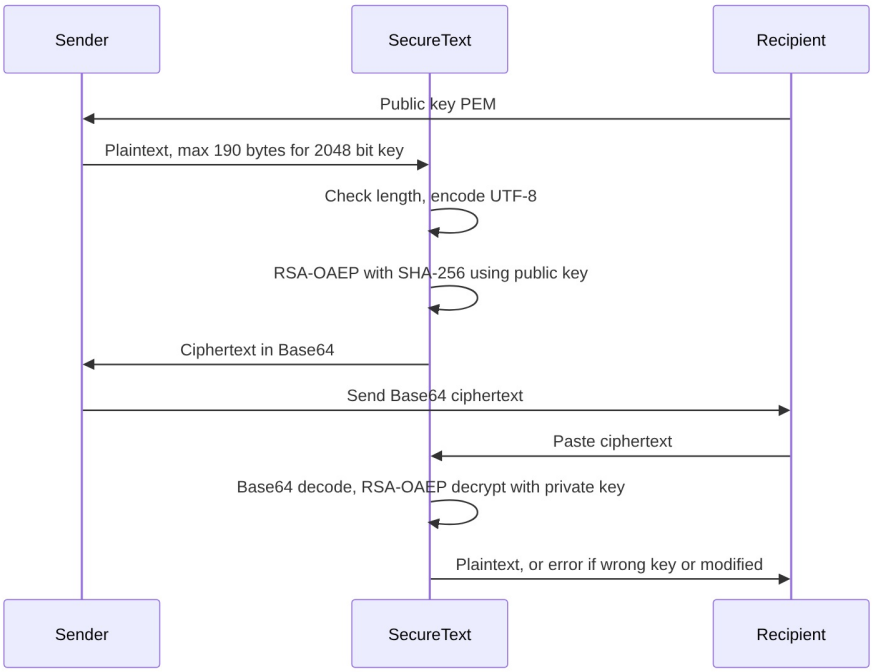
24. RSA Encryption and Decryption Algorithms

Encryption (RSAES-OAEP, SHA-256, MGF1-SHA-256):

1. Encode the message as UTF-8 bytes; reject empty input.
2. If length > k − 2·32 − 2 (190 bytes for 2048-bit keys) show an error recommending hybrid mode.
3. Apply OAEP encoding with a fresh random seed, then the RSA public operation.
4. Base64-encode the k-byte ciphertext.

Decryption:

1. Base64-decode (strict); reject non-Base64 or wrong length.
2. Apply the RSA private operation and OAEP decoding; on failure report “wrong private key or corrupted ciphertext”.
3. Decode UTF-8.

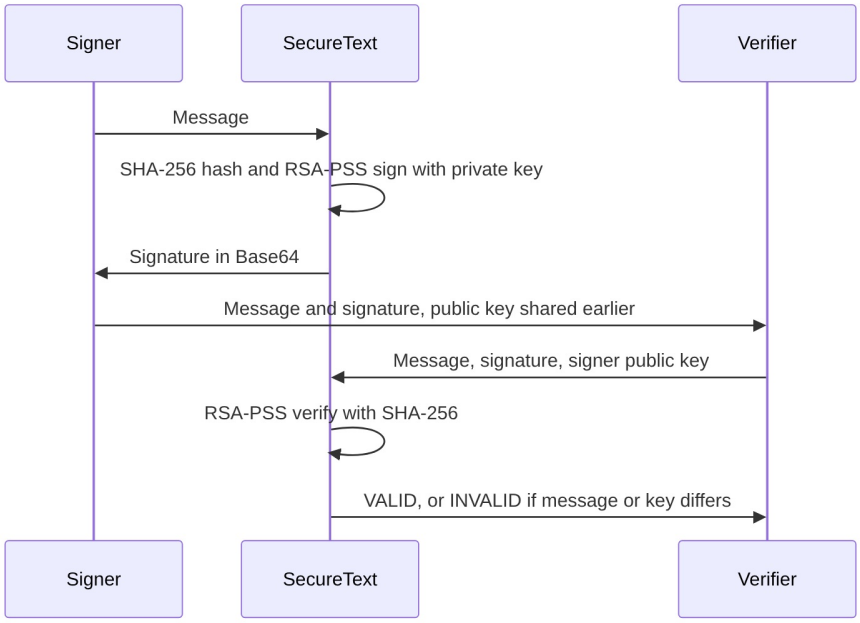


Encryption sequence

25. Digital-Signature Algorithm

Sign (RSASSA-PSS, SHA-256, MGF1-SHA-256, salt length 32): hash the UTF-8 message; encode with PSS using a random salt; apply the RSA private operation; Base64-encode.

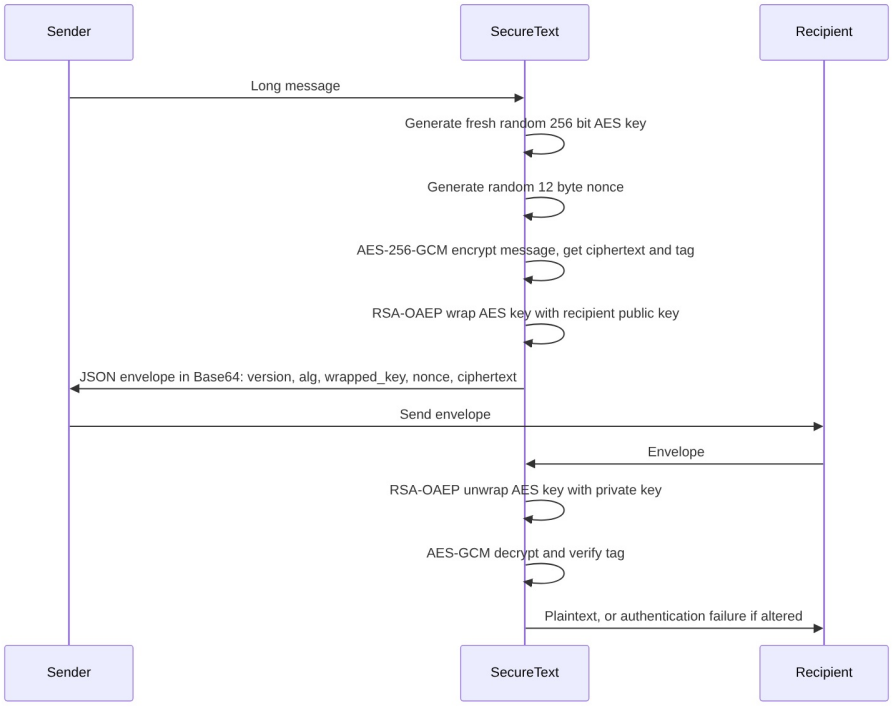
Verify: Base64-decode; apply the public operation; check PSS encoding against the hash of the received message. Output True/False. Malformed signatures return False rather than crashing.



Signature sequence

26. Hybrid-Encryption Workflow

1. Generate a fresh 256-bit AES key and a random 96-bit nonce.
2. Encrypt with AES-256-GCM; associated data is `1|RSA-OAEP-SHA256+AES-256-GCM` so the header is authenticated.
3. Wrap the AES key with RSA-OAEP.
4. Output Base64(JSON{version, alg, wrapped_key, nonce, ciphertext}) where ciphertext includes the 16-byte tag.
5. Decrypt: unwrap the key, check key length, AES-GCM decrypt; any modification raises an authentication error.



Hybrid workflow

Because a new key is generated for every message, a random nonce never repeats under the same key, which satisfies the GCM uniqueness requirement [5].

27. Detailed Module Descriptions

Module	Responsibility

`main.py`	Checks Tkinter and `cryptography`, starts the GUI
`crypto/rsa_keys.py`	Key generation; PEM export/import (public: SubjectPublicKeyInfo; private: encrypted PKCS#8); fingerprint; numeric access for the tutorial
`crypto/rsa_encryption.py`	OAEP encrypt/decrypt, size limit, tamper demo helper
`crypto/digital_signatures.py`	PSS sign and verify
`crypto/hybrid_encryption.py`	AES-GCM + RSA-OAEP envelope, tamper demo helper
`crypto/toy_rsa.py`	Educational textbook RSA with tiny primes
`utils/validation.py`	Non-empty text, password length (>= 8), key size (>= 2048)
`utils/file_handler.py`	UTF-8 text IO; private files written with mode 0600 where supported
`gui/main_window.py`	Eight tabs: Dashboard, Key Management, Encryption, Decryption, Digital Signature, Hybrid Encryption, RSA Math, About & Help
`gui/content.py`	Tutorial and help text
`tests/`	Unit tests and a GUI-wiring check

28. Implementation Details

- **Language and libraries:** Python 3; Tkinter/ttk (standard library); `cryptography` (OpenSSL backend).
- **Error handling:** the crypto layer raises specific exceptions (`ValidationError`, `DecryptionError`, `KeyError_`); the GUI catches them and shows a coloured status message. Wrong password, invalid PEM, malformed Base64 and corrupted envelopes never produce a traceback.
- **Secrets:** no hardcoded passwords or keys; passwords are requested through masked dialogs, are not logged, and are dropped after use; the RSA tutorial displays only bit lengths of p, q and d, never their values.
- **State:** keys are kept in memory; the user chooses whether to export them.
- **Recipient key:** an imported public key becomes the recipient/verifier key; otherwise the user's own public key is used so the app can be tested alone.

29. Important Code Explanations

OAEP padding and size limit (`crypto/rsa_encryption.py`)

```
        def oaep_padding():
            return padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()),
                                algorithm=hashes.SHA256(), label=None)

        def max_message_bytes(public_key):
            k = (public_key.key_size + 7) // 8
            return k - 2 * HASH_LEN - 2
```

For 2048 bits, k = 256 so the limit is 256 − 64 − 2 = 190 bytes.

PSS signing (`crypto/digital_signatures.py`)

```
        def pss_padding():
            return padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                               salt_length=padding.PSS.DIGEST_LENGTH)
```

Hybrid encryption (`crypto/hybrid_encryption.py`)

```
        aes_key = AESGCM.generate_key(bit_length=256)
        nonce = os.urandom(NONCE_BYTES)
        ciphertext = AESGCM(aes_key).encrypt(nonce, message.encode("utf-8"), _AAD)
        wrapped = public_key.encrypt(aes_key, oaep_padding())
```

Protected private-key export (`crypto/rsa_keys.py`)

```
        private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.BestAvailableEncryption(password.encode("utf-8")))
```

30. Testing Methodology

- **Unit tests** (`unittest`, 46 cases, IDs T01 to T46) for key handling, OAEP, PSS, hybrid mode and the toy RSA, including negative tests (wrong key, tampering, malformed input, oversize, empty input, Unicode, invalid key files).
- **GUI wiring check** (`tests/gui_logic_check.py`, 30 checks) that drives every GUI handler against a simulated Tkinter module.
- **Manual GUI test** following `docs/demo_script.md`: to be performed on the target Windows machine.
- RSA key generation is cached across tests to keep the suite fast.

31. Test-Case Table

ID	Description	Input	Expected output	Status
T01	RSA key generation (2048-bit)	generate_key_pair(2048)	2048-bit modulus, e = 65537	PASS
T02	Reject weak / invalid key size	1024 and ‘abc’	ValidationError raised	PASS
T03	RSA math relationships on a real key	p, q, n, e, d of a generated key	$pq = n$; $ed = 1 \bmod \text{lcm}(p-1,q-1)$; $(m^e)^d \bmod n = m$	PASS
T04	Public key PEM export/import	export then load (bytes and str)	Loaded key equals original	PASS
T05	Private key export is encrypted and re-importable	password ‘correct horse battery’	PEM header ENCRYPTED PRIVATE KEY; key restored	PASS
T06	Wrong password on private key import	wrong password	KeyError_ raised	PASS
T07	Short / empty export password	‘short’ and ’’	ValidationError raised	PASS
T08	Invalid key file content	garbage bytes, empty bytes	KeyError_ raised	PASS
T09	Public key file given as private key	public PEM + password	KeyError_ raised	PASS
T10	Private key file save and load via file handler	temp file in new sub-folder	File created; key restored	PASS
T11	Fingerprint stable and distinct	two different keys	Same key same fingerprint; different keys differ	PASS
T12	RSA-OAEP encrypt/decrypt round trip	‘Hello SecureText’	Original text recovered; plaintext not in ciphertext	PASS
T13	Ciphertext is Base64 of key-size bytes	‘abc’	256 bytes after Base64 decode	PASS
T14	OAEP is randomised	same message encrypted twice	Different ciphertexts, both decrypt	PASS
T15	Decrypt with incorrect private key	key A encrypts, key B decrypts	DecryptionError raised	PASS
T16	Invalid ciphertext handling	non-Base64, short, all-zero blocks	DecryptionError raised	PASS
T17	Tampered ciphertext	one bit flipped	DecryptionError raised	PASS
T18	Oversized direct RSA message	190 bytes OK, 191 bytes	Limit 190; 191 rejected with hint to use Hybrid tab	PASS
T19	Unicode message	Hindi, emoji, accented, Chinese text	Exact text recovered	PASS
T20	Limit counts UTF-8 bytes not characters	96 x ‘é’ (192 bytes)	ValidationError raised	PASS
T21	Empty and invalid input	‘‘,’ ’, None, 123	ValidationError raised	PASS
T22	Whitespace/newlines in pasted ciphertext	ciphertext wrapped at 64 columns	Decrypts correctly	PASS
T23	Signature creation and valid verification	‘Pay Alice 100’	256-byte signature; verification True	PASS
T24	Verification after message modification	‘Pay Alice 900’ with original signature	Verification False	PASS
T25	Verification with wrong public key	key B public key	Verification False	PASS
T26	PSS signatures are randomised	same message signed twice	Different signatures, both verify	PASS

T27	Corrupted / malformed signature	flipped byte, '###', short Base64	Verification False	PASS
T28	Empty signature inputs	empty message / empty signature	ValidationError raised	PASS
T29	Unicode signature	Hindi + symbol text; text plus trailing space	Valid, then invalid after change	PASS
T30	Signature does not hide message	'visible text'	Signature does not contain text (message stays public)	PASS
T31	Hybrid round trip, long message	~12,000 characters	Original text recovered	PASS
T32	Hybrid envelope structure	'hi'	5 documented fields; nonce 12 B; wrapped key 256 B; ciphertext = 2 + 16 B tag	PASS
T33	Fresh AES key and nonce each time	same message twice	Different nonce, wrapped key and ciphertext	PASS
T34	Altered hybrid ciphertext detected	one bit flipped in ciphertext	DecryptionError (authentication failed)	PASS
T35	Altered GCM tag detected	last byte flipped	DecryptionError raised	PASS
T36	Altered nonce / wrapped key detected	one bit flipped each	DecryptionError raised	PASS
T37	Unsupported envelope version	version = 2	DecryptionError raised	PASS
T38	Hybrid with wrong private key	key A encrypts, key B decrypts	DecryptionError raised	PASS
T39	Malformed envelopes	bad Base64, non-JSON, list, missing fields, short nonce	DecryptionError raised	PASS
T40	Unicode hybrid	Tamil, emoji, Japanese text	Exact text recovered	PASS
T41	Hybrid empty input	'','', None	ValidationError raised	PASS
T42	Direct-RSA tamper demo helper	tamper_ciphertext then decrypt	DecryptionError raised	PASS
T43	Hybrid tamper demo helper	tamper_envelope then decrypt	DecryptionError raised	PASS
T44	Toy RSA textbook example	p=61, q=53, e=17, m=65	n=3233, phi=3120, d=2753, c=2790, recovered 65	PASS
T45	Toy RSA invalid parameters	non-prime p, p=q, bad e, m too big, non-int	ToyRSAError raised	PASS
T46	Primality helper	numbers 0..19	2,3,5,7,11,13,17,19	PASS

Status values come from the recorded run (Section 32). The full table with the Actual-output column is in `docs/test_cases.md`.

32. Actual Results

Recorded run (full output in `docs/test_results.txt`):

Item	Value
Environment	Python 3.13.17, Linux, `cryptography` 50.0.1
Command	`python -m unittest discover -s tests -v`
Tests run	46
Passed	46
Failed	0
GUI handler checks (fake Tkinter)	30 of 30 passed

Not verified in the development environment: real Tkinter rendering and mouse interaction (no Tkinter or display available), Windows behaviour, and performance. Re-run the tests on your own machine and record your own output.

33. Screenshots

Screenshots could not be captured in the development environment. Insert your own after running the application; the list of recommended captures is in `screenshots/README.md`.

Figure	Content	Status
Fig. A	Dashboard / key generation	[insert]
Fig. B	Encryption and decryption	[insert]
Fig. C	Wrong-key demonstration	[insert]
Fig. D	Signature valid and tampered	[insert]
Fig. E	Hybrid encryption	[insert]
Fig. F	Terminal showing test results	[insert]

34. Results and Discussion

- Encryption followed by decryption with the matching key recovers the original text, including Unicode text (T12, T19).
- Ciphertexts of the same message differ each time (T14) and signatures differ each time (T26), showing the effect of OAEP and PSS randomisation.
- Wrong private keys and modified ciphertext fail with clear errors (T15, T17, T38).
- Messages above 190 bytes are rejected by direct RSA with a helpful message, while hybrid mode handles a message of about 12,000 characters (T18, T31).
- Any change to the hybrid ciphertext, tag, nonce, wrapped key or header is detected (T34 to T37).
- Signatures verify for the original message and fail for a modified message or a different public key (T23 to T25).
- Private-key files are encrypted; a wrong password is rejected (T05, T06).

35. Security Analysis

Threat	Mitigation in SecureText	Residual risk
Eavesdropping	RSA-OAEP / AES-256-GCM	Compromise of private key or endpoint
Ciphertext modification	OAEP validation; GCM tag	None for detection within the algorithms' guarantees
Message forgery	RSA-PSS signatures	Depends on correct public key
Chosen-plaintext guessing	Randomised OAEP	None practical
Weak keys	Minimum 2048 bits enforced; e = 65537	Future factoring or quantum advances
Stolen key file	Password-encrypted PKCS#8	Weak passwords; minimum length is only 8
Nonce reuse in GCM	Fresh key and random nonce per message	None practical
Public-key substitution (man-in-the-middle)	SHA-256 fingerprint displayed	Users must compare fingerprints out of band; no certificates
Side channels, malware, memory scraping	Library implementations	Not addressed in the application

The application does not add forward secrecy, replay protection or key revocation.

36. Advantages

- Uses standard, well-reviewed algorithms through an established library.
- Works offline with one dependency.
- Clear separation of GUI, cryptography and utilities, which makes the logic testable.
- Demonstrates failure behaviour as well as success.
- Educational tutorial that distinguishes the toy example from the real implementation.

37. Limitations

- No certificates or public-key authentication.
- No key rotation, revocation or backup service.
- Keys are in memory and are not securely wiped.
- Text only; no file or network features.
- The password strength check is only a length check.
- Not independently audited; not for protecting real sensitive data.

- The real GUI was not run in the development sandbox (see Section 32).
- RSA is not quantum-resistant.

38. Real-World Applications

TLS certificates and key exchange, code signing, S/MIME and PGP e-mail, SSH authentication, document signing and JSON web tokens. Hybrid encryption (asymmetric key protection plus symmetric bulk encryption) is the basic pattern behind secure communication protocols.

39. Future Enhancements

- X.509 certificates and trust validation.
- Elliptic-curve (ECDH/ECDSA) options and post-quantum algorithms.
- File and attachment encryption.
- Network messaging between two instances.
- Key rotation, expiry and a keystore with OS-level protection.
- Password-strength meter and a memory-hardened key-derivation setting.

40. Conclusion

SecureText demonstrates how RSA is used correctly in practice: key sizes of at least 2048 bits, OAEP for encryption, PSS for signatures, a hybrid scheme for long messages, and encrypted private-key storage. The automated tests (46 of 46 passed in the recorded run) and the failure demonstrations show that the system rejects wrong keys, modified data and invalid input. The project's remaining gaps, in particular public-key authentication and key lifecycle management, are documented rather than hidden.

41. References

References marked (V) had their bibliographic details confirmed through an online search during preparation; the others were written from standard knowledge and should be checked against your library or the publisher's page before submission.

1. W. Diffie and M. E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644-654, 1976.
2. R. L. Rivest, A. Shamir and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120-126, 1978.
3. K. Moriarty (Ed.), B. Kaliski, J. Jonsson and A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2," RFC 8017, IETF, November 2016. https://www.rfc-editor.org/rfc/rfc8017 (V)
4. M. Bellare and P. Rogaway, "Optimal Asymmetric Encryption," in *Advances in Cryptology - EUROCRYPT '94*, Springer, 1995.
5. M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, 2007. https://csrc.nist.gov/pubs/sp/800/38/d/final (V)
6. E. Barker, "Recommendation for Key Management: Part 1 - General," NIST SP 800-57 Part 1 Revision 5, 2020. https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final (V)
7. NIST, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.
8. W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed., Pearson. https://www.pearson.com/store/p/cryptography-and-network-security-principles-and-practice/P200000003477 (V, edition listing)
9. J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed., CRC Press, 2020.
10. A. Menezes, P. van Oorschot and S. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.
11. Python Software Foundation, *The Python Standard Library: unittest and tkinter*, https://docs.python.org/3/
12. Python Cryptographic Authority, *cryptography documentation*, https://cryptography.io/

42. Appendix: Installation and Execution

Windows 10/11 (terminal inside the project folder that contains `main.py`):

```
python --version
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

```
python main.py
python -m unittest discover -s tests -v
python tests\gui_logic_check.py
```

Troubleshooting: see `README.md` (missing packages, wrong folder, Tkinter not installed, file permissions, password-protected key import).

Converting this report to PDF: `pandoc docs/project_report.md -o report.pdf --pdf-engine=xelatex --resource-path=docs`, run from the project root. A pre-built `docs/SecureText_Project_Report.pdf` is included.